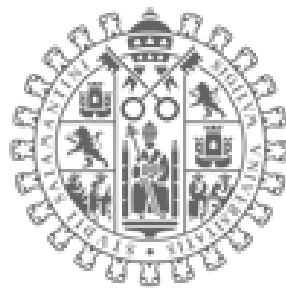


Sistemas Inteligentes

Naive Bayes



**VNiVERSiDAD
D SALAMANCA**

CAMPUS OF INTERNATIONAL EXCELLENCE

Miguel Barranquero Diez
Diego Fernández Sutil

Índice

1. Introducción	3
2. Fundamento teórico	4
2.1. Teorema de Bayes y criterio MAP	4
2.2. Clasificador bayesiano Naive	4
2.3. Naive Bayes Gaussiano para atributos continuos	4
2.4. Ventajas e inconvenientes	5
3. Conjunto de datos Palmer Penguins	6
3.1. Origen y motivación	6
3.2. Descripción de las variables	6
3.3. Variable objetivo y desequilibrio de clases	6
3.4. Atributos utilizados y valores faltantes	7
3.5. Separabilidad esperada	7
4. Implementación en C	8
4.1. Carga y preprocesado del dataset	8
4.2. Entrenamiento: estimación de medias, varianzas y priors	9
4.3. Inferencia en log-espacio	10
5. Metodología experimental	11
5.1. Validación	11
5.2. Métricas	11
6. Resultados	12
6.1. Modelo entrenado	12
6.2. Evaluación hold-out 70 / 30	12
6.3. Validación cruzada k -fold	12
6.4. Ejemplo individual de predicción	13
7. Discusión	14
8. Conclusiones	15
Bibliografía	15

1. Introducción

El razonamiento bayesiano constituye uno de los pilares del aprendizaje automático probabilístico. Su idea fundamental consiste en asignar a las hipótesis una probabilidad que representa el grado de creencia y, a medida que se observan nuevos datos, actualizar esa creencia mediante el teorema de Bayes [1].

Dentro de este marco, el **clasificador bayesiano Naive** es uno de los métodos más utilizados en la práctica: pese a la simplicidad de la hipótesis de independencia que asume, obtiene un rendimiento sorprendentemente competitivo, en muchos casos comparable al de las redes neuronales o los árboles de decisión [2].

El objetivo de esta práctica es:

- Implementar desde cero, en lenguaje C, un clasificador bayesiano Naive Gaussiano.
- Aplicarlo al conjunto de datos **Palmer Penguins** [3], [4], un dataset de morfometría de pingüinos antárticos ampliamente utilizado como banco de pruebas para clasificación supervisada.
- Realizar los ajustes de preprocesado (tratamiento de valores ausentes) y validación (partición estratificada y validación cruzada k -fold) para estimar de forma robusta su rendimiento.
- Elaborar un informe con las conclusiones derivadas de los resultados obtenidos.

2. Fundamento teórico

2.1. Teorema de Bayes y criterio MAP

Dado un conjunto de hipótesis H y un conjunto de datos observados D , el teorema de Bayes permite calcular la probabilidad a posteriori de cada hipótesis:

$$P(h | D) = \frac{P(D | h)P(h)}{P(D)}$$

donde $P(h)$ es la probabilidad a priori, $P(D|h)$ la verosimilitud y $P(D)$ la evidencia.

Para tomar una decisión, el criterio del **Máximo a Posteriori** (MAP) selecciona la hipótesis más probable a la luz de los datos. Como $P(D)$ es constante respecto a h , basta con maximizar el producto verosimilitud por prior:

$$h_{\text{MAP}} = \arg \max_{h \in H} P(h | D) = \arg \max_{h \in H} P(D | h)P(h)$$

2.2. Clasificador bayesiano Naive

Cuando el objetivo es clasificar una instancia $\mathbf{x} = \langle a_1, \dots, a_n \rangle$ en alguna clase $v_j \in V$, el criterio MAP se concreta en:

$$v_{\text{MAP}} = \arg \max_{v_j \in V} P(v_j | a_1, \dots, a_n) = \arg \max_{v_j \in V} P(a_1, \dots, a_n | v_j)P(v_j)$$

La estimación directa de $P(a_1, \dots, a_n | v_j)$ es inviable, ya que el número de combinaciones posibles crece exponencialmente con n . Para hacerla tratable, Naive Bayes introduce la **hipótesis de independencia condicional** entre atributos dada la clase:

$$P(a_1, \dots, a_n | v_j) = \prod_{i=1}^n P(a_i | v_j)$$

Sustituyendo en la regla MAP, se obtiene el clasificador Naive Bayes:

$$v_{\text{NB}} = \arg \max_{v_j \in V} P(v_j) \prod_{i=1}^n P(a_i | v_j)$$

2.3. Naive Bayes Gaussiano para atributos continuos

Los cuatro atributos morfométricos del dataset Palmer Penguins (longitud y profundidad del culmen, longitud de la aleta y masa corporal) son **continuos**, por lo que no es posible estimar $P(a_i | v_j)$ por simple recuento. La estrategia habitual consiste en asumir que, condicionado a la clase, cada atributo se distribuye según una **normal** $\mathcal{N}(\mu_{cj}, \sigma_{cj}^2)$ y estimar sus parámetros por máxima verosimilitud sobre el conjunto de entrenamiento:

$$\mu_{cj} = \frac{1}{n_c} \sum_{i: y_i = c} x_{ij}, \quad \sigma_{cj}^2 = \frac{1}{n_c - 1} \sum_{i: y_i = c} (x_{ij} - \mu_{cj})^2$$

La verosimilitud de un valor x bajo esa hipótesis viene dada por la densidad de la normal:

$$p(x | v_j) = \frac{1}{\sqrt{2\pi\sigma_{cj}^2}} \exp\left(-\frac{(x - \mu_{cj})^2}{2\sigma_{cj}^2}\right)$$

Para evitar problemas numéricos derivados del producto de muchas probabilidades pequeñas, la clasificación se realiza en **log-espacio**:

$$\log P(v_j | \mathbf{x}) \propto \log P(v_j) + \sum_{i=1}^n \log p(a_i | v_j)$$

2.4. Ventajas e inconvenientes

Como se discutió en la teoría de la asignatura, Naive Bayes presenta las siguientes **ventajas**: es simple, rápido, robusto frente al ruido, requiere relativamente pocos ejemplos para entrenar y proporciona de forma natural la probabilidad estimada de cada predicción. Sus principales **inconvenientes** son la propia hipótesis de independencia (rara vez se cumple estrictamente), la necesidad de asumir una forma de distribución para atributos continuos y el hecho de que las probabilidades estimadas tienden a estar mal calibradas, aunque la clasificación final sea correcta [1].

3. Conjunto de datos Palmer Penguins

3.1. Origen y motivación

El dataset **Palmer Penguins** [3], [4] recopila medidas morfométricas de pingüinos antárticos recogidas entre 2007 y 2009 por la estación Palmer LTER, en la península Antártica, dentro del programa *Long Term Ecological Research* de la *National Science Foundation* de Estados Unidos. Fue popularizado como conjunto de datos didáctico por Allison Horst en el paquete `palmerpenguins` para R.

Su elección como banco de pruebas para esta práctica responde a varios motivos: (i) es un dataset no trivial, con clases desbalanceadas y valores faltantes que obligan a preprocesar; (ii) mezcla variables continuas, categóricas y temporales, lo que permite discutir el tratamiento de cada tipo; y (iii) presenta tres especies con grados de solapamiento morfológico distintos, lo que da lugar a una matriz de confusión informativa.

3.2. Descripción de las variables

El fichero `raw_penguin_dataset.csv` está delimitado por punto y coma (;) y contiene 344 instancias y 11 columnas. La cabecera completa, ordenada según el fichero, es:

Columna	Variable	Tipo	Descripción
1	<code>individual_id</code>	Identificador	Código único del individuo.
2	<code>clutch_completion</code>	Categórica binaria	¿Completó la puesta? <code>Yes</code> / <code>No</code> .
3	<code>date_egg</code>	Temporal (fecha)	Fecha en que se observó el huevo.
4	<code>culmen_length_mm</code>	Númerica continua	Longitud del culmen (pico), en mm.
5	<code>culmen_depth_mm</code>	Númerica continua	Profundidad del culmen, en mm.
6	<code>flipper_length_mm</code>	Númerica continua	Longitud de la aleta, en mm.
7	<code>body_mass_g</code>	Númerica continua	Masa corporal, en gramos.
8	<code>sex</code>	Categórica	<code>MALE</code> / <code>FEMALE</code> / <code>NA</code> .
9	<code>delta_15_N</code>	Númerica continua	Isótopo $\delta^{15}\text{N}$ en sangre.
10	<code>delta_13_C</code>	Númerica continua	Isótopo $\delta^{13}\text{C}$ en sangre.
11	<code>species</code>	Categórica (clase)	<code>Adelie</code> / <code>Chinstrap</code> / <code>Gentoo</code> .

Table 1: Esquema del fichero `raw_penguin_dataset.csv` . La columna 11 (`species`) actúa como variable objetivo.

3.3. Variable objetivo y desequilibrio de clases

La variable a predecir es `species` , que toma tres valores correspondientes a otras tantas especies del género *Pygoscelis*:

- **Adelie** (*Pygoscelis adeliae*): 152 ejemplares en bruto.
- **Chinstrap** (*Pygoscelis antarcticus*): 68 ejemplares.
- **Gentoo** (*Pygoscelis papua*): 124 ejemplares.

El conjunto está **claramente desbalanceado**: Chinstrap representa únicamente un 19.8% de las instancias, frente al 44.2% de Adelie. Este desbalanceo impone dos consideraciones metodológicas: (i) la partición train/test debe ser **estratificada** para preservar la proporción en ambos subconjuntos, y (ii) los priors $P(c)$ del clasificador ya **no son uniformes**, lo que matiza la regla MAP del Naive Bayes (la clase minoritaria necesita más evidencia para ‘ganar’).

3.4. Atributos utilizados y valores faltantes

De los 11 campos disponibles se han retenido los cuatro atributos morfométricos clásicamente empleados con este dataset [4]:

- `culmen_length_mm`
- `culmen_depth_mm`
- `flipper_length_mm`
- `body_mass_g`

Las columnas `individual_id`, `clutch_completion`, `date_egg` y `sex` se descartan por ser identificadores, indicadores categóricos o temporales que requerirían un tratamiento específico distinto a la asunción gaussiana del modelo. Las firmas isotópicas `delta_15_N` y `delta_13_C` también se descartan porque contienen un número apreciable de NAs y aportan información ecológica que va más allá del objetivo de la práctica.

El dataset contiene valores faltantes codificados como `NA`. En particular, 2 filas tienen NA en alguno de los cuatro atributos morfométricos seleccionados. La estrategia aplicada es la más sencilla y conservadora: **eliminar la fila completa** mediante un filtrado en el momento de la carga. Como sólo dos ejemplos se ven afectados sobre 344, la pérdida es despreciable y evita tener que imputar valores. Tras este filtrado el conjunto queda reducido a 342 instancias con la distribución de clases siguiente:

Clase	N. ejemplos	Proporción
Adelie	151	44.15%
Chinstrap	68	19.88%
Gentoo	123	35.97%
Total	342	100.00%

Table 2: Distribución de clases tras filtrar las filas con NA en los atributos morfométricos.

3.5. Separabilidad esperada

Desde el punto de vista biológico, *Gentoo* es la especie más distintiva: presenta una masa corporal considerablemente mayor (en torno a 5 kg frente a los 3.7 kg de las otras dos especies) y un culmen menos profundo. *Adelie* y *Chinstrap*, por su parte, tienen tamaños globales similares, pero se diferencian sobre todo en la longitud del culmen (notablemente mayor en *Chinstrap*). Por tanto, cabe esperar que el clasificador distinga *Gentoo* con casi total facilidad y concentre los errores en la frontera entre *Adelie* y *Chinstrap*, que es precisamente lo que reflejan los resultados experimentales presentados más adelante.

4. Implementación en C

La implementación se ha realizado en **C estándar** (C99), sin dependencias externas más allá de la biblioteca matemática `<math.h>`. El programa completo se encuentra en el fichero `naive_bayes.c` que acompaña a este informe. La estructura general es la siguiente:

```
typedef struct {
    double features[N_FEATURES];
    int class_id;
} Sample;

typedef struct {
    double prior[N_CLASSES];
    double mean[N_CLASSES][N_FEATURES];
    double var[N_CLASSES][N_FEATURES];
} NBModel;
```

4.1. Carga y preprocesado del dataset

El CSV utiliza punto y coma como delimitador y mezcla campos con NA. Se ha escrito un pequeño tokenizador en sitio que parte cada línea en sus 11 campos, descarta las filas con NA en alguno de los cuatro atributos morfométricos, traduce la etiqueta textual a un identificador entero mediante el diccionario `class_names[]` y rellena la estructura `Sample`.

```
static int load_csv(const char *path, Sample *data, int *skipped_na) {
    FILE *f = fopen(path, "r");
    char line[MAX_LINE];
    fgets(line, sizeof(line), f); /* descarta la cabecera */
    int n = 0, na = 0;
    while (fgets(line, sizeof(line), f) && n < MAX_SAMPLES) {
        /* recorta saltos de línea y espacios al final */
        size_t L = strlen(line);
        while (L > 0 && (line[L-1] == '\n' || line[L-1] == '\r'
                        || line[L-1] == ' ' || line[L-1] == '\t'))
            line[--L] = '\0';
        if (L == 0) continue;

        /* tokeniza por ';' in-place */
        char *fields[N_CSV_FIELDS] = {0};
        int nf = 0; char *tok = line;
        for (char *p = line; p < line + L; p++) {
            if (*p == ';' || *p == '\0') {
                int last = (*p == '\0'); *p = '\0';
                if (nf < N_CSV_FIELDS) fields[nf++] = tok;
                if (last) break;
                tok = p + 1;
            }
        }
        if (nf < N_CSV_FIELDS) continue;

        /* descarta filas con NA en los 4 atributos morfométricos */
        if (strcmp(fields[3], "NA") == 0 || strcmp(fields[4], "NA") == 0 ||
            strcmp(fields[5], "NA") == 0 || strcmp(fields[6], "NA") == 0) {
            na++; continue;
        }
        int cid = get_class_id(fields[10]);
        if (cid < 0) continue;
    }
}
```

```

    data[n].features[0] = atof(fields[3]);
    data[n].features[1] = atof(fields[4]);
    data[n].features[2] = atof(fields[5]);
    data[n].features[3] = atof(fields[6]);
    data[n].class_id = cid;
    n++;
}
fclose(f);
if (skipped_na) *skipped_na = na;
return n;
}

```

Sólo se aplica una técnica de preprocesado: **eliminación de filas con NA** en alguno de los cuatro atributos seleccionados (2 filas de 344). No se ha aplicado normalización: Naive Bayes Gaussiano modela cada atributo con su propia μ y σ^2 , de modo que es invariante a cambios de escala lineales.

4.2. Entrenamiento: estimación de medias, varianzas y priors

Para cada clase c se calcula el prior $P(c)$ como su frecuencia relativa y, para cada atributo j , la media μ_{cj} y la varianza muestral σ_{cj}^2 por máxima verosimilitud. Como medida de robustez numérica, se aplica un **suavizado mínimo de varianza** ($1e-9$) que evita divisiones por cero en caso de que un atributo tomase un único valor para una clase:

```

static void train_nb(NBModel *m, const Sample *train, int ntrain) {
    int counts[N_CLASSES] = {0};
    double sum[N_CLASSES][N_FEATURES] = {{0}};
    double sumsq[N_CLASSES][N_FEATURES] = {{0}};
    for (int i = 0; i < ntrain; i++) {
        int c = train[i].class_id;
        counts[c]++;
        for (int j = 0; j < N_FEATURES; j++)
            sum[c][j] += train[i].features[j];
    }
    for (int c = 0; c < N_CLASSES; c++) {
        m->prior[c] = (double)counts[c] / ntrain;
        for (int j = 0; j < N_FEATURES; j++)
            m->mean[c][j] = sum[c][j] / counts[c];
    }
    for (int i = 0; i < ntrain; i++) {
        int c = train[i].class_id;
        for (int j = 0; j < N_FEATURES; j++) {
            double d = train[i].features[j] - m->mean[c][j];
            sumsq[c][j] += d * d;
        }
    }
    for (int c = 0; c < N_CLASSES; c++)
        for (int j = 0; j < N_FEATURES; j++) {
            m->var[c][j] = sumsq[c][j] / (counts[c] - 1);
            if (m->var[c][j] < 1e-9) m->var[c][j] = 1e-9;
        }
}

```

4.3. Inferencia en log-espacio

La predicción se realiza acumulando logaritmos de probabilidades, lo que evita el underflow asociado al producto de cuatro densidades gaussianas. Tras obtener los $\log P(v_j | \mathbf{x})$ se aplica una normalización tipo softmax para devolver las probabilidades posteriores normalizadas:

```
static double log_gaussian(double x, double mean, double var) {
    return -0.5 * log(2.0 * M_PI * var)
        - 0.5 * (x - mean) * (x - mean) / var;
}

static int predict(const NBModel *m, const double *x, double *posterior) {
    double log_p[N_CLASSES];
    double max_lp = -INFINITY;
    for (int c = 0; c < N_CLASSES; c++) {
        log_p[c] = log(m->prior[c]);
        for (int j = 0; j < N_FEATURES; j++)
            log_p[c] += log_gaussian(x[j], m->mean[c][j], m->var[c][j]);
        if (log_p[c] > max_lp) max_lp = log_p[c];
    }
    double sum = 0;
    for (int c = 0; c < N_CLASSES; c++) {
        posterior[c] = exp(log_p[c] - max_lp);
        sum += posterior[c];
    }
    int best = 0;
    for (int c = 0; c < N_CLASSES; c++) {
        posterior[c] /= sum;
        if (posterior[c] > posterior[best]) best = c;
    }
    return best;
}
```

5. Metodología experimental

5.1. Validación

Para estimar el rendimiento del clasificador se han combinado dos esquemas de validación complementarios:

1. **Partición estratificada hold-out 70 / 30.** Tras barajar la muestra con semilla fija (`srand(42)`) para garantizar reproducibilidad, se reserva el 70% de los ejemplos de cada clase para entrenamiento (238 instancias) y el 30% restante para test (104 instancias). La estratificación es especialmente importante en este dataset desbalanceado: garantiza que también en el conjunto de test cada clase aparezca en su proporción real (46 Adelie, 21 Chinstrap, 37 Gentoo) y evita estimaciones sesgadas.
2. **Validación cruzada estratificada k -fold**, con $k = 5$ y $k = 10$. Cada fold contiene aproximadamente la misma proporción de ejemplos de cada clase, y el modelo se reentrena desde cero en cada iteración. La media y la desviación típica de la accuracy proporcionan una estimación robusta del rendimiento esperado.

5.2. Métricas

Para cada experimento se reportan:

- La **matriz de confusión**, que permite identificar qué clases se confunden entre sí.
- La **accuracy global**, definida como la fracción de aciertos.
- **Precision, recall** y **F1-score** por clase, junto con su promedio macro.

6. Resultados

6.1. Modelo entrenado

Tras entrenar sobre el conjunto de 238 ejemplos de la partición train, los parámetros estimados son los recogidos en la Table 3. Los priors reflejan el desbalanceo del dataset: $P(\text{Adelie}) = 0.4412$, $P(\text{Chinstrap}) = 0.1975$, $P(\text{Gentoo}) = 0.3613$. Las medias por clase reproducen el comportamiento biológico esperado: *Gentoo* destaca por una masa corporal notablemente mayor (5073 g) y un culmen menos profundo (15 mm); *Chinstrap* tiene el culmen más largo (49 mm) frente a *Adelie* (38.7 mm), pero ambas especies presentan una masa corporal muy similar.

Clase	culmen_length	culmen_depth	flipper_length	body_mass
Adelie	38.73 ± 2.63	18.28 ± 1.20	189.90 ± 6.55	3686 ± 445
Chinstrap	49.00 ± 3.51	18.45 ± 1.12	196.11 ± 6.93	3729 ± 372
Gentoo	47.90 ± 3.22	15.07 ± 1.02	217.64 ± 6.84	5073 ± 526

Table 3: Media ± desviación típica por clase y atributo estimadas sobre el conjunto de entrenamiento (semilla 42). Unidades: mm para culmen y aleta, g para masa.

6.2. Evaluación hold-out 70 / 30

Aplicando el modelo al conjunto de test (104 ejemplos: 46 Adelie, 21 Chinstrap, 37 Gentoo) se obtiene la matriz de confusión de la Table 4. La accuracy global sobre test es del **97.12 %**.

Real \ Predicha	Adelie	Chinstrap	Gentoo
Adelie	44	2	0
Chinstrap	1	20	0
Gentoo	0	0	37

Table 4: Matriz de confusión sobre el conjunto de test (104 ejemplos).

Los errores se concentran, como era de esperar, en la frontera *Adelie/Chinstrap*: 2 Adelie clasificadas como Chinstrap y 1 Chinstrap clasificada como Adelie. La clase *Gentoo* se clasifica con un acierto perfecto, en línea con la fuerte separabilidad morfológica anticipada. Las métricas por clase y su promedio macro se resumen en la Table 5.

Clase	Precision	Recall	F1-score
Adelie	0.9778	0.9565	0.9670
Chinstrap	0.9091	0.9524	0.9302
Gentoo	1.0000	1.0000	1.0000
Macro promedio	0.9623	0.9696	0.9658

Table 5: Precision, recall y F1-score por clase sobre el conjunto de test.

6.3. Validación cruzada k -fold

Aunque la partición 70 / 30 ofrece una estimación inicial, una sola partición puede ser optimista o pesimista en función de cómo caigan los ejemplos. Para obtener una estimación más robusta, se ha repetido la evaluación con validación cruzada estratificada con $k = 5$ y $k = 10$. Los resultados se resumen en la Table 6.

Esquema	Accuracy media	Desv. típica	Mín. / Máx.
Hold-out 70 / 30	0.9712	—	—
5-fold CV	0.9679	0.0110	0.9552 / 0.9851
10-fold CV	0.9708	0.0183	0.9412 / 1.0000

Table 6: Resumen comparativo de los esquemas de validación empleados.

Las tres estimaciones son consistentes entre sí (accuracy en torno al **97 %**), con una desviación típica muy baja en el caso 5-fold (1.1 %), lo que indica que el modelo no es sensible a la partición concreta de los datos. La desviación típica algo mayor en 10-fold se debe a que cada fold contiene sólo 33–36 ejemplos, por lo que un único error desplaza la accuracy del fold en torno al 3 %.

6.4. Ejemplo individual de predicción

Para ilustrar el funcionamiento del clasificador, se presenta la salida ante una instancia conocida del dataset (un Adelie con culmen 39.1 mm, profundidad 18.7 mm, aleta 181 mm y masa 3750 g):

```

Instancia [39.1, 18.7, 181.0, 3750.0]
Probabilidades posteriores:
P(Adelie | x) = 0.9981
P(Chinstrap | x) = 0.0019
P(Gentoo | x) = 0.0000
Prediccion: Adelie

```

El modelo asigna la práctica totalidad de la probabilidad posterior a la clase correcta, aunque deja un pequeño residuo (0.19 %) para *Chinstrap*, que es la clase morfológicamente más cercana. Como se señaló en la sección teórica, esta tendencia a producir probabilidades extremas es típica de Naive Bayes y debe interpretarse con cautela: la clasificación es correcta, pero la magnitud exacta de la probabilidad no es una estimación calibrada.

7. Discusión

Los resultados obtenidos están en línea con los reportados habitualmente para este dataset y este tipo de clasificador [4], [5]. El error se concentra en la frontera entre *Adelie* y *Chinstrap*, donde el solapamiento entre las distribuciones de culmen y masa corporal es real y no atribuible al modelo. Algunas observaciones:

- **La hipótesis de independencia condicional es claramente incorrecta** en este dataset (la longitud del culmen y la longitud de la aleta están claramente correlacionadas con la masa corporal), pero el clasificador funciona bien igualmente. Esto refuerza la idea de la teoría: Naive Bayes no necesita probabilidades perfectamente calibradas, sino que la clase correcta obtenga simplemente la mayor puntuación.
- **La hipótesis Gaussiana es razonable** para los atributos de Palmer Penguins (medidas morfométricas dentro de una especie), lo que ayuda. En dominios donde la distribución condicional fuese fuertemente no normal cabría esperar un descenso de rendimiento, que se aliviaría mediante discretización o modelos de mezcla.
- **El desbalanceo de clases es manejable** gracias a que el modelo incorpora los priors $P(c)$ aprendidos de los datos. La clase minoritaria (*Chinstrap*, 19.9 %) obtiene un recall del 95 %, lo que indica que el prior penaliza poco a la clase pequeña porque las verosimilitudes la separan bien.
- **El coste computacional es despreciable**. El entrenamiento es prácticamente lineal en el número de ejemplos y atributos, $O(nd)$, y la inferencia $O(cd)$ por ejemplo.

8. Conclusiones

A partir de esta práctica se pueden extraer las siguientes conclusiones:

1. Se ha implementado en C, desde cero, un clasificador Naive Bayes Gaussiano completo, con partición estratificada y validación cruzada k -fold.
2. Aplicado al dataset Palmer Penguins (342 ejemplos tras filtrar NAs, 3 clases desbalanceadas), el clasificador alcanza una **accuracy en torno al 97 %**, consistente entre hold-out, 5-fold y 10-fold, con errores localizados exclusivamente en la frontera *Adelie* / *Chinstrap* y un acierto perfecto sobre *Gentoo*.
3. Pese a la simplicidad del modelo (4 medias, 4 varianzas y un prior por clase, 27 parámetros en total) y al hecho de que la hipótesis de independencia condicional no se cumple, Naive Bayes resulta competitivo y muy eficiente, lo que confirma el análisis teórico discutido en clase.
4. El uso de cálculos en log-espacio y el suavizado mínimo de varianza son detalles prácticos relevantes para una implementación numéricamente estable. El tratamiento explícito de valores ausentes (filtrado de NAs) ilustra que, incluso en un dataset ‘académico’, el preprocesado mínimo es ineludible.

En conjunto, la práctica ilustra cómo un modelo con supuestos simplificadores ‘ingenuos’ puede seguir siendo una elección válida y razonable como *baseline* en problemas de clasificación con atributos continuos.

Bibliografía

- [1] T. M. Mitchell, *Machine Learning*. New York: McGraw-Hill, 1997.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ: Pearson, 2010.
- [3] K. B. Gorman, T. D. Williams, and W. R. Fraser, “Ecological Sexual Dimorphism and Environmental Variability within a Community of Antarctic Penguins (Genus *Pygoscelis*),” *PLOS ONE*, vol. 9, no. 3, p. e90081, 2014, doi: 10.1371/journal.pone.0090081.
- [4] A. M. Horst, A. P. Hill, and K. B. Gorman, “palmerpenguins: Palmer Archipelago (Antarctica) penguin data.” [Online]. Available: <https://allisonhorst.github.io/palmerpenguins/>
- [5] scikit-learn developers, “Naive Bayes classifiers.” [Online]. Available: https://scikit-learn.org/stable/modules/naive_bayes.html